

Lecture 1: Course Introduction

Formal Verification

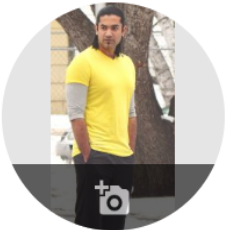
CSU44004/CSU55004

Dr. Saif U. R. Malik
Assistant Professor, SCSS
sumalik@tcd.ie
G.40 O'Reily Building

Saif Ur Rehman Malik

- ❖ From Islamabad, Pakistan
- ❖ PhD (2011-2014) in Computer Science, North Dakota State Univ., USA
Using Formal Methods to Validate the Usage, Protocols, and Feasibility in Large Scale Computing Systems
- ❖ Asst. Professor COMSATS University Pakistan
Assoc. Professor (Ten.), COMSATS University Pakistan (2014-2019)
- ❖ Senior Researcher, Cybernetica, Tallinn Estonia (2019-2024) – Trustworthy Systems
- ❖ Asst. Professor in Computer Science, SCSS, TCD (2024) – Federated Learning (Machine learning with privacy)

Google Scholar Research Profile



Saif U. R. Malik

FOLLOW

Trinity College Dublin

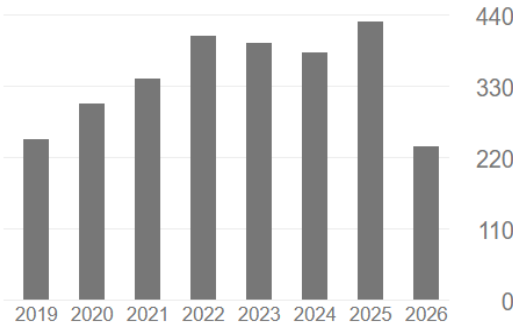
Verified email at tcd.ie - [Homepage](#)

[Formal Methods](#) [Model Checking](#) [Data Security and Privacy](#) [Cloud](#) [Fog and Edge Computing](#)

☐	TITLE			CITED BY	YEAR
☐	Digital Twin Monitoring System for Chronic Disease Patients: A Simulation-Based Proof of Concept B Abbas, SR Malik, MA Shah, MI Khan, G Srivastava, SA Moqurrab, ... Computational and Structural Biotechnology Journal 35 (2), 0103				2026
☐	Ensuring zero trust in gdpr-compliant deep federated learning architecture Z Abbas, SF Ahmad, A Anjum, MH Syed, SUR Malik, S Rehman Computers 14 (8), 317			9	2025
☐	Towards Sustainable IoT: A Digital Signature-Enhanced Federated Learning Approach SH Qureshi, SUR Malik, J Haseeb, SA Moqurrab, TFN Bukht, ... Security and Privacy 8 (4), e70066			2	2025
☐	IoV-TwinChain: Predictive maintenance of vehicles in internet of vehicles through digital twin and blockchain M Iqbal, S Suhail, R Matulevičius, FA Shah, SUR Malik, K McLaughlin Internet of Things 30, 101514			31	2025

Cited by [VIEW ALL](#)

	All	Since 2021
Citations	3477	2206
h-index	33	28
i10-index	70	56



Public access [VIEW ALL](#)

1 article	18 articles
not available	available

Software has become critical to modern life

- ❖ Process Control (oil, gas, water, . . .)
- ❖ Transportation (air traffic control, . . .)
- ❖ Health Care (patient monitoring, device control . . .)
- ❖ Finance (automatic trading, bank security . . .)
- ❖ Defense (intelligence, weapons control, . . .)
- ❖ Manufacturing (precision milling, assembly, . . .)

Embedded Software



Failing Software Costs Money

- ❖ Thousands of dollars for each minute of factory down-time
- ❖ Huge losses of monetary and intellectual investment
 - Rocket boost failure (e.g., Ariane 5, European Space Agency spent **10 years and \$7 billion** to produce Ariane 5, a giant rocket capable of hurling a pair of three-ton satellites into orbit)

A Heart Device Is Found Vulnerable to Hacker Attacks – New York Times

www.nytimes.com/2008/03/12/busin Reader

HOME PAGE MY TIMES TODAY'S PAPER VIDEO MOST POPULAR TIMES TOPICS

The New York Times **Business**

WORLD U.S. N.Y. / REGION BUSINESS TECHNOLOGY SCIENCE HEALTH SPORTS OPINION

MEDIA & ADVERTISING WORLD BUSINESS SMALL BUSINESS YOUR MONEY DEALBOOK MARKETS RESI

A Heart Device Is Found Vulnerable to Hacker Attacks

By [BARNABY J. FEDER](#)
Published: March 12, 2008

To the long list of objects vulnerable to attack by computer hackers, add the human heart.

The threat seems largely theoretical. But a team of computer security researchers plans to report Wednesday that it had been able to gain wireless access to a combination heart defibrillator and pacemaker.

They were able to reprogram it to shut down and to deliver jolts of electricity that would potentially be fatal — if the device had been in a person. In this case, the researcher were hacking into a device in a laboratory.

[TWITTER](#)

[LINKEDIN](#)

[SIGN IN TO E-MAIL OR SAVE THIS](#)

[PRINT](#)

[REPRINTS](#)

[SHARE](#)

The peculiarity of software systems

Tiny faults can have catastrophic consequences

- ❖ Software seems particularly prone to faults:
 - Ariane 5
 - Mars Climate Orbiter,
 - London Ambulance Dispatch System
 - Denver Airport Luggage Handling System
 - Pentium-Bug
 - (more at <http://www5.in.tum.de/~huckle/bugse.html>)

Software Complexity

- ❖ No physical or modal separation of subsystems
- ❖ Local failures often affect whole system
- ❖ Software designs have very high logic complexity
- ❖ Cost efficiency more important than reliability

HOW TO ENSURE SOFTWARE CORRECTNESS?

A Central Strategy: Testing

- ❖ Limitations of Testing
- ❖ Testing can show the presence of errors, but not their absence

(exhaustive testing viable only for trivial systems)

- ❖ Representativeness of test cases/injected faults is selective.

Unexpected/Rare cases?

Formal Methods (FMs)

❖ What are Formal Methods?

Rigorous mathematically-based techniques and tools for the specification, development, and verification of software and hardware systems.

❖ Why Formal Methods?

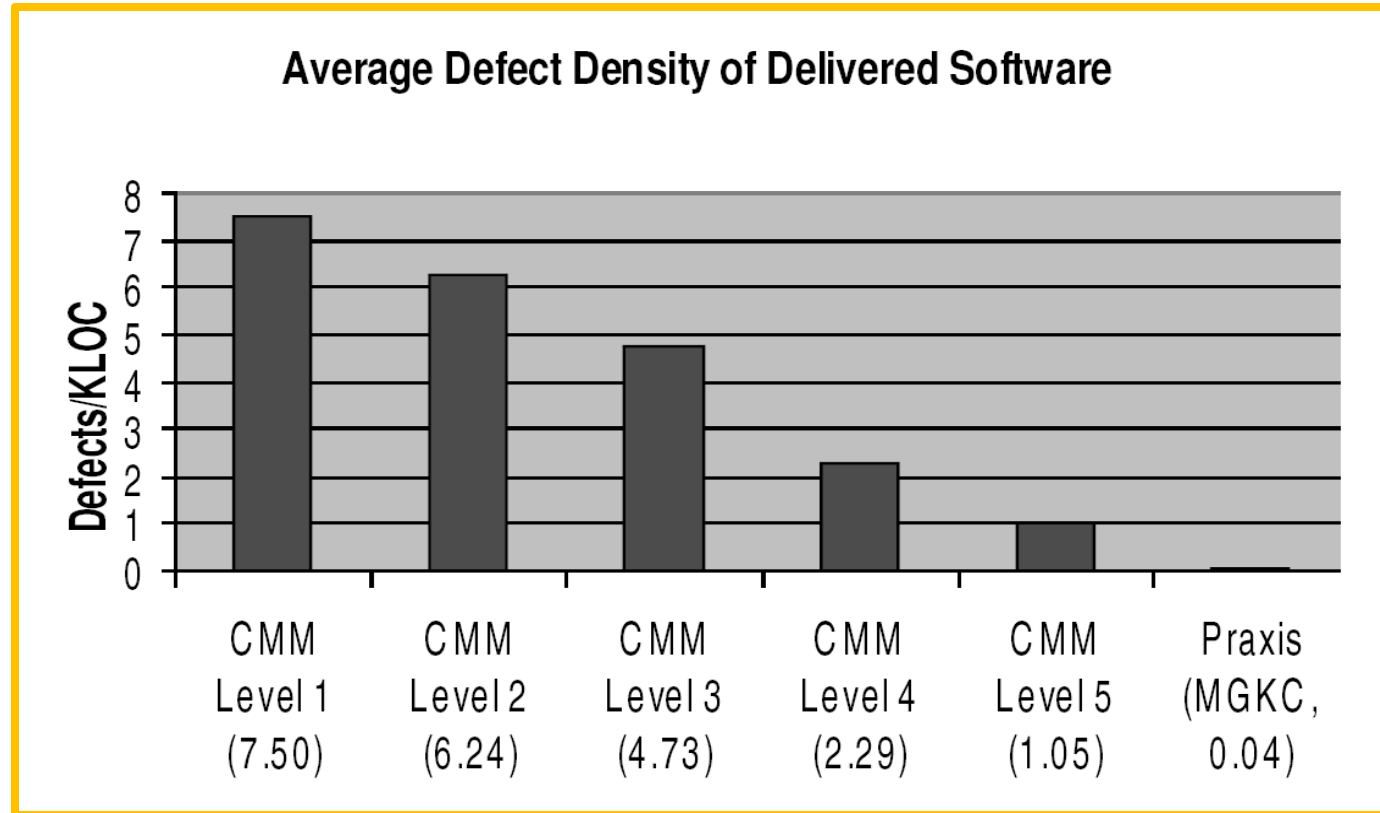
- They lead to zero defect software... **NO!**
- They are better than testing... **NO!**
- They prove that computer scientists are highly skilled mathematicians... **NO!**

The Myths

- ❖ Hence you should know the myths¹:
 1. FMs can guarantee software is perfect
 2. FMs are all about program proving
 3. FMs are only useful for safety-critical systems
 4. FMs require highly trained mathematicians
 5. FMs increase the cost of development
 6. FMs are unacceptable to users
 7. FMs are not used on real, large scale software

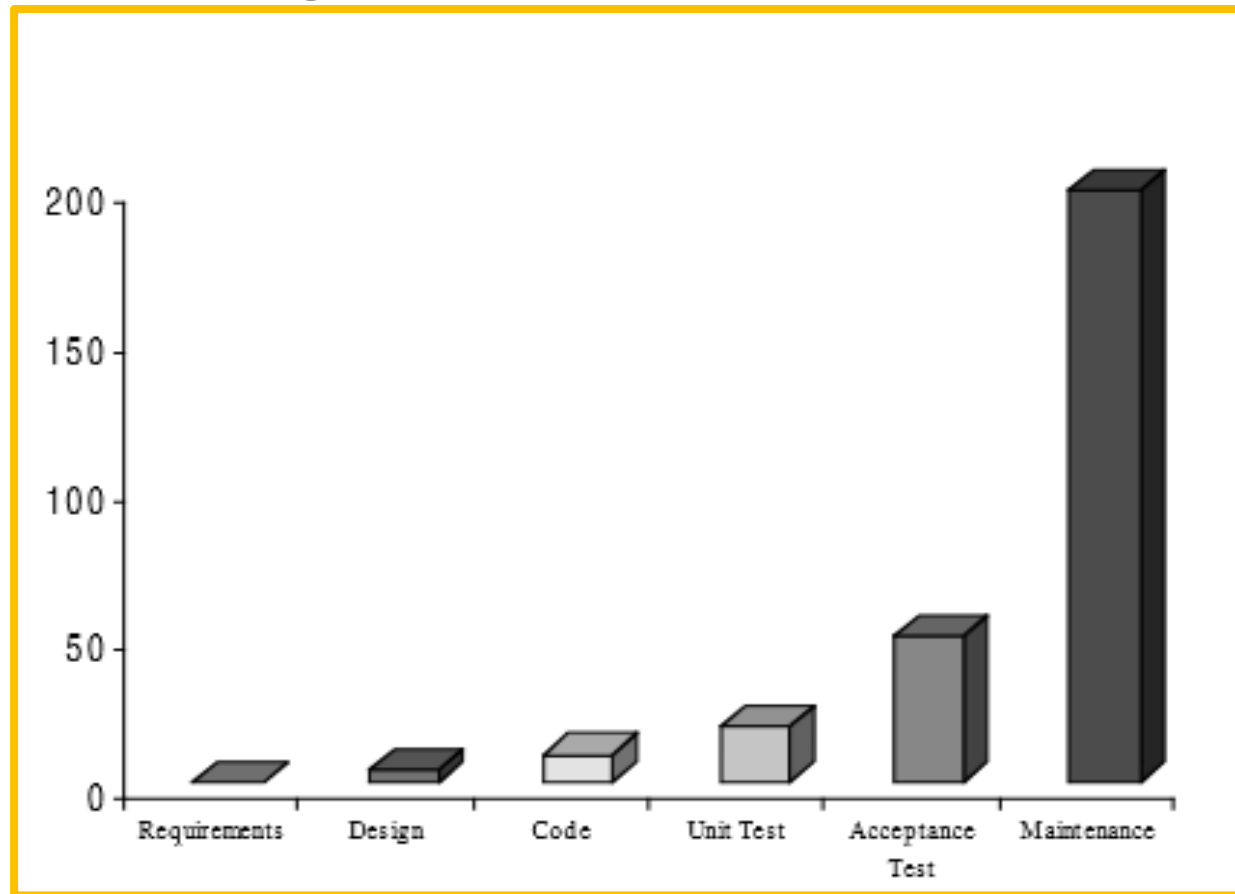
The Reality

Very Low Defect Rates (But Not Zero)



The Reality (cont'd)

cost of correcting a requirements defect according to the stage at which it is discovered



The Reality (cont'd)



Fallible

易出错

证明并不比测试更能保证正确性,在许多情况下,其可靠程度甚至远不如测试。

- Proofs are no more a guarantee of correctness than testing, in many cases far less of one.



Cost Effective

- Lead to early error discovery...
- This reduces the overall costs...



Straightforward Math

- To write a formal specification in Z requires basic knowledge of set theory and logic... (high-school)

Formal Verification (FV)

- ❖ Formal verification is a mathematical technique used to prove that a system, program, hardware design, or algorithm behaves exactly according to its specification.
- ❖ Ultimate goal: prove the absence of software errors.

Methods of FV

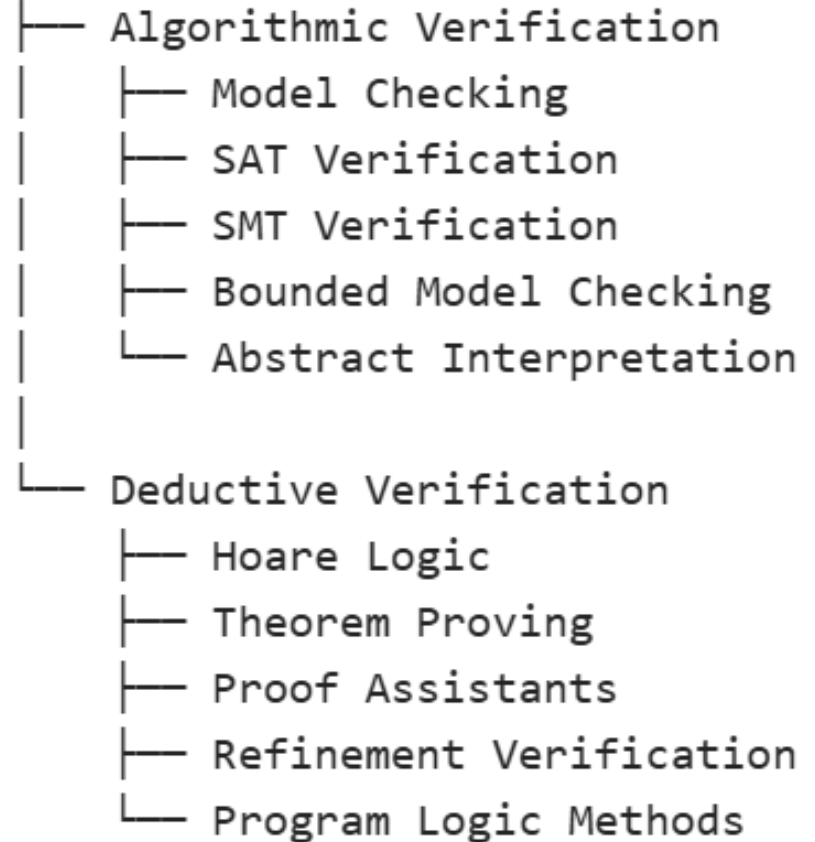
❖ Algorithmic Verification

- ❖ uses algorithms/programs to automatically determine whether a system/model satisfies a specification.

❖ Deductive Verification

- ❖ relies on mathematical proofs. Instead of exploring states automatically, the verifier proves that the implementation satisfies a formal specification.

Formal Verification



Holy Grail of Formal Verification

- ❖ **Sound:** If the verification method returns yes, then indeed the program under examination has no bug
- ❖ **Complete:** If the verification method returns no, then indeed the program under examination has a bug
- ❖ **Terminating:** The verification method always terminates, returning yes or no.

***Sound and Terminating** systems can prove the correctness of virtually every program we would care about.*

This Module

❖ **Deductive verification**

- ❖ create a correctness proof of the program in a logic (with axioms and logical rules)
- ❖ We will manually prove programs correctness using pen and paper proofs.
- ❖ We will use software that provide some automation to make this easier.

This Module

❖ CSU44004/CSU55004 Theory

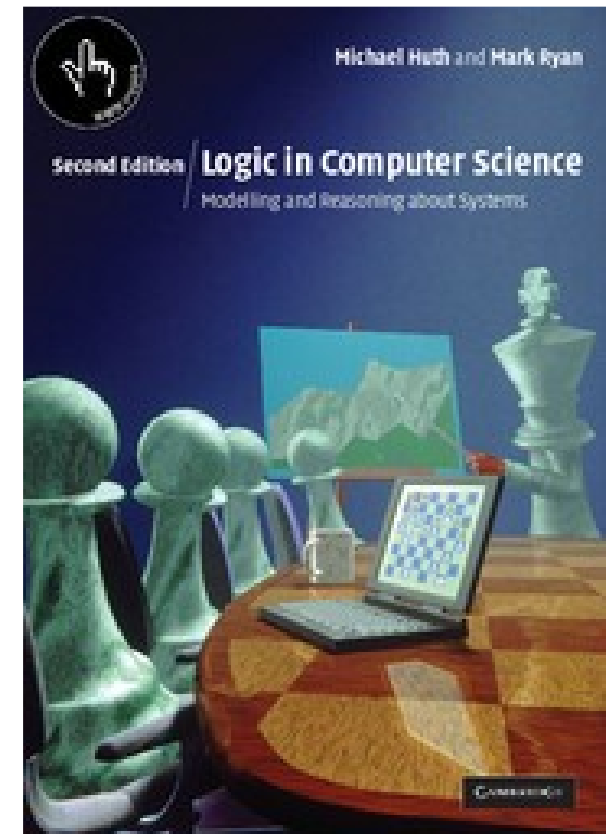
❖ **Book:** Logic in Computer Science: Modelling and Reasoning about Systems, 2nd Edition, by M. Huth & M. Ryan

❖ 1. Symbolic logic

- ❖ Natural deduction
- ❖ Propositional logic
- ❖ First-order Predicate logic

❖ 2. Correctness of imperative programs

- ❖ Floyd-Hoare logic
- ❖ Weakest Precondition calculus
- ❖ Loop invariants
- ❖ functional abstractions, etc.



This Module

❖ CSU44004/CSU55004: Practice

Exercises, Tutorials and Assignments:

- ❖ Pencil & paper proofs: propositional, inductive, correctness proofs.
- ❖ Semi-automatic proofs using the Dafny verifier

This Module

❖ CSU44004/CSU55004: Other

Marks:

- 35% marks from coursework
- 65% marks from annual exam (before Christmas)
- Second attempt supplementals: 100% exam

See Module Descriptor

- ❖ Final exam: pencil & paper proofs of simple propositional properties, and program correctness properties.
- ❖ You will need to install VS Code + the Dafny extension.
- ❖ Two lectures per week – mix of new material and in-class tutorial exercises.
- ❖ All information about the module on blackboard.